UNIVERSIDAD TECNOLÓGICA EMILIANO ZAPATA

RAÚL CORCINO LEÓN

ANDRÉS HERNÁNDEZ POPOCA

BRYAN VENEGAS SÁNCHEZ

ALEJANDRO CASTELAR HERNÁNDEZ

OSCAR ROSALES GARCÍA

T.S.U. - INFRAESTRUCTURA DE REDES DIGITALES

Ecuaciones Diferenciales

Tarea Integradora

Profesor: Edvaart Sethaziz Jatirian Foltides

5to Cuatrimestre

abril de 2026

ÍNDICE

1. INTRODUCCIÓN 4

1.1 Objetivo del Proyecto 4

1.2 Competencias Desarrolladas 4

1.3 Tecnologías Utilizadas 4

2. EXPLICACIÓN DEL MÉTODO MATEMÁTICO 5

2.1 Transformada de Laplace 5

2.2 Aplicación a Ecuaciones Diferenciales 5

Paso 1: Aplicar la transformada de Laplace 5

Paso 2: Resolver algebraicamente en el dominio de Laplace 5

Paso 3: Aplicar la transformada inversa 5

2.3 Ventajas del Método 5

3. ARQUITECTURA DEL SISTEMA 6

3.1 Arquitectura General 6

3.2 Flujo de Datos 6

4. ESTRUCTURA DEL CÓDIGO 8

4.1 Organización de Archivos 8

4.2 Módulo Principal (app.py) 8

4.3 Módulo Solver (utils/solver.py) 9

4.4 Componentes del Frontend 10

5. INSTALACIÓN Y CONFIGURACIÓN 11

5.1 Requisitos Previos 11

5.2 Instalación en Windows 11

5.3 Instalación en macOS/Linux 11

5.4 Acceso a la Aplicación 12

6. FUNCIONAMIENTO DEL PROGRAMA 13

6.1 Interfaz Principal 13

6.2 Tutorial Interactivo 13

6.3 Procesamiento de Ecuaciones 14

6.4 Presentación de Resultados 14

6.5 Persistencia de Datos 15

6.6 Modo Oscuro 15

6.7 Manejo de Errores 16

7. EJEMPLOS RESUELTOS 17

7.1 Caso 1: y' + 2y = 0, y(0) = 1 17

7.2 Caso 2: y'' + 3y' + 2y = 0, y(0) = 0, y'(0) = 1 18

7.3 Caso 3: y' - y = e^t, y(0) = 1 18

7.4 Caso 4: y'' + y = sin(t), y(0) = 0, y'(0) = 0 19

8. REFLEXIÓN FINAL 21

8.1 ¿Qué dificultades se encontraron al implementar matemáticas en código? 21

1. Parsing de expresiones matemáticas 21
2. Manejo de símbolos y sustituciones 21
3. Transformada inversa de Laplace 21
4. Visualización numérica de soluciones simbólicas 21
5. Validación de ecuaciones lineales 22

8.2 ¿Cómo se mejoraría el programa? 22

9. CONCLUSIONES 24

9.1 Logros del Proyecto 24

9.2 Competencias Adquiridas 24

9.3 Aplicaciones Prácticas 25

9.4 Trabajo Futuro 25

CRÉDITOS 26

1. INTRODUCCIÓN

1.1 Objetivo del Proyecto

El presente proyecto tiene como objetivo desarrollar una aplicación web en Python capaz de resolver ecuaciones diferenciales ordinarias de primer y segundo orden utilizando la transformada de Laplace. La aplicación integra conceptos matemáticos avanzados con técnicas modernas de desarrollo web, proporcionando una interfaz amigable para estudiantes y profesionales.

1.2 Competencias Desarrolladas

Durante el desarrollo del proyecto se aplicaron las siguientes competencias:

- **Métodos matemáticos computacionales**: Implementación de la transformada de Laplace y su inversa
- **Algoritmos en Python**: Diseño de código modular y reutilizable
- **Interpretación de resultados**: Presentación clara de soluciones matemáticas
- **Documentación técnica**: Código comentado y estructurado

1.3 Tecnologías Utilizadas

- **Backend**: Flask 3.1.0
- **Cálculo simbólico**: SymPy 1.13.3
- **Cálculo numérico**: NumPy 2.2.4
- **Visualización**: Matplotlib 3.10.1
- **Frontend**: HTML5, CSS3, JavaScript (ES6+)
- **Renderizado matemático**: MathJax 3

2. EXPLICACIÓN DEL MÉTODO MATEMÁTICO

2.1 Transformada de Laplace

La transformada de Laplace es una herramienta matemática que convierte funciones del dominio del tiempo t al dominio de la frecuencia compleja s. Se define como:

$L\{f(t)\} = F(s) = \int_0^\infty f(t)e^{(-st)}\, dt$

2.2 Aplicación a Ecuaciones Diferenciales

Para resolver una ecuación diferencial ordinaria, el método sigue estos pasos:

Paso 1: Aplicar la transformada de Laplace

Para derivadas, se utilizan las siguientes propiedades:

- $L\{y'(t)\} = sY(s) - y(0)$
- $L\{y''(t)\} = s^2Y(s) - sy(0) - y'(0)$

Paso 2: Resolver algebraicamente en el dominio de Laplace

La ecuación diferencial se convierte en una ecuación algebraica en términos de Y(s), que se puede resolver mediante manipulación algebraica.

Paso 3: Aplicar la transformada inversa

Una vez obtenida Y(s), se aplica la transformada inversa de Laplace para obtener la solución final y(t):

$y(t) = L^{-1}\{Y(s)\}$

2.3 Ventajas del Método

- Convierte ecuaciones diferenciales en ecuaciones algebraicas más simples
- Incorpora automáticamente las condiciones iniciales
- Es especialmente útil para ecuaciones lineales con coeficientes constantes
- Permite trabajar con funciones discontinuas y delta de Dirac

3. ARQUITECTURA DEL SISTEMA

3.1 Arquitectura General

El sistema implementa una arquitectura web cliente-servidor basada en el patrón MVC (Model-View-Controller):

```
┌──────────────────────────────────────────────────┐
│                CLIENTE (Navegador)                 │
│  ┌──────────┐  ┌──────────┐  ┌──────────────┐    │
│  │   HTML   │  │   CSS    │  │  JavaScript  │    │
│  │ (Vistas) │  │ (Estilos)│  │   (Lógica)   │    │
│  └──────────┘  └──────────┘  └──────────────┘    │
└──────────────────────────────────────────────────┘
                      ↕ HTTP
┌──────────────────────────────────────────────────┐
│            SERVIDOR (Flask/Python)                 |
│  ┌──────────────────────────────────────────┐     │
│  │         app.py (Controlador)             │     │
│  └──────────────────────────────────────────┘     │
│  ┌──────────────────────────────────────────┐     │
│  │    utils/solver.py (Modelo/Lógica)       │     │
│  │    - Parsing de ecuaciones               │     │
│  │    - Transformada de Laplace             │     │
│  │    - Transformada inversa                │     │
│  │    - Generación de gráficas              │     │
│  └──────────────────────────────────────────┘     │
└──────────────────────────────────────────────────┘
```

3.2 Flujo de Datos

1. El usuario ingresa la ecuación y condiciones iniciales en el formulario web
2. El navegador envía una petición POST al servidor Flask
3. El servidor procesa la ecuación usando SymPy
4. Se genera la solución matemática y la gráfica
5. Los resultados se renderizan en formato LaTeX con MathJax

6. La respuesta HTML se envía de vuelta al cliente

4. ESTRUCTURA DEL CÓDIGO

4.1 Organización de Archivos

```
utez_laplace_web/
├── app.py                    # Aplicación Flask principal
├── requirements.txt          # Dependencias del proyecto
├── README.md                # Documentación general
├── static/                  # Archivos estáticos
│   ├── css/
│   │   └── styles.css       # Estilos de la interfaz
│   ├── js/
│   │   └── app.js          # Lógica del cliente
│   └── img/
│       └── logo_UTEZ.png   # Logo institucional
├── templates/               # Plantillas HTML
│   ├── base.html           # Plantilla base
│   ├── index.html          # Página principal
│   └── _tutorial.html      # Modal del tutorial
├── tests/                   # Pruebas unitarias
│   └── test_solver.py      # Tests del solver
└── utils/                   # Módulos utilitarios
    ├── __init__.py
    └── solver.py           # Lógica de resolución
```

4.2 Módulo Principal (app.py)

Este archivo contiene la aplicación Flask y define dos rutas principales:

Ruta principal (/)

- Métodos: GET, POST
- Función: Renderiza el formulario y procesa las ecuaciones
- Maneja errores de parsing y cálculo

Características principales:

python

- Validación de entrada del usuario

- Manejo de excepciones personalizado (SolverError)

- Renderizado de plantillas con contexto
- Persistencia de datos del formulario

4.3 Módulo Solver (utils/solver.py)

Este módulo contiene toda la lógica matemática del proyecto:

Funciones principales:

1. **parse_linear_equation(equation_text)**
    - Normaliza el texto de entrada
    - Identifica el orden de la ecuación
    - Extrae coeficientes y término forzante
    - Valida que sea lineal con coeficientes constantes
2. **build_laplace_solution(parsed, y0, y1)**
    - Aplica la transformada de Laplace
    - Resuelve para Y(s)
    - Aplica la transformada inversa
    - Retorna la solución simbólica
3. **plot_solution(solution_expr)**
    - Evalúa numéricamente la solución
    - Genera gráfica con Matplotlib
    - Codifica imagen en base64
    - Maneja números complejos
4. **solve_with_laplace(equation_text, y0_text, y1_text)**
    - Función principal que orquesta el proceso completo
    - Retorna diccionario con todos los resultados

4.4 Componentes del Frontend

HTML (templates/index.html)

- Formulario de captura con validación
- Sección de resultados con renderizado LaTeX
- Tarjetas de ejemplos predefinidos
- Sistema de tutorial interactivo

CSS (static/css/styles.css)

- Sistema de temas claro/oscuro
- Variables CSS para colores institucionales
- Diseño responsive con CSS Grid
- Animaciones y transiciones suaves

JavaScript (static/js/app.js)

- Persistencia con localStorage
- Control del tutorial modal
- Toggle de tema oscuro
- Auto-rellenado de ejemplos

5. INSTALACIÓN Y CONFIGURACIÓN

5.1 Requisitos Previos

- Python 3.10 o superior
- pip (gestor de paquetes de Python)
- Navegador web moderno

5.2 Instalación en Windows

Desde powershell

```
# 1. Clonar o descargar el proyecto
cd ruta/del/proyecto

# 2. Crear entorno virtual
python -m venv .venv

# 3. Activar entorno virtual
.\.venv\Scripts\Activate.ps1

# 4. Instalar dependencias
pip install -r requirements.txt

# 5. Ejecutar la aplicación
python app.py
```

5.3 Instalación en macOS/Linux

```
# 1. Navegar al directorio del proyecto
cd ruta/del/proyecto

# 2. Crear entorno virtual
python3 -m venv .venv

# 3. Activar entorno virtual
source .venv/bin/activate

# 4. Instalar dependencias
pip install -r requirements.txt

# 5. Ejecutar la aplicación
```

python app.py

5.4 Acceso a la Aplicación

Una vez iniciado el servidor, abrir el navegador en:

http://127.0.0.1:5000

6. FUNCIONAMIENTO DEL PROGRAMA

6.1 Interfaz Principal

Al acceder a la aplicación, el usuario encuentra:

Encabezado:

- Logo institucional de UTEZ
- Título del proyecto
- Descripción breve de la funcionalidad
- Botón para abrir tutorial
- Toggle de modo oscuro

Panel de Entrada:

- Campo de texto para la ecuación diferencial
- Campo para y(0)
- Campo para y'(0) (solo para segundo orden)
- Botón "Resolver ecuación"
- Botón "Borrar contenido"

Casos de Prueba Predefinidos: Cuatro tarjetas clickeables con los ejemplos requeridos:

1. y' + 2y = 0, y(0) = 1
2. y'' + 3y' + 2y = 0, y(0) = 0, y'(0) = 1
3. y' - y = e^t, y(0) = 1
4. y'' + y = sin(t), y(0) = 0, y'(0) = 0

6.2 Tutorial Interactivo

Al primer acceso, se muestra un modal tutorial con 4 pasos:

1. **Escribir la ecuación**: Formato aceptado y ejemplos
2. **Agregar condiciones iniciales**: Explicación de y(0) y y'(0)

3. **Resolver**: Qué resultados se mostrarán
4. **Características adicionales**: Funcionalidades del sistema

6.3 Procesamiento de Ecuaciones

Formatos aceptados:

y' + 2y = 0

y'' + 3y' + 2y = 0

y' - y = e^t

y'' + y = sin(t)

y'' + 4y' + 4y = exp(t)

Funciones soportadas:

- Exponenciales: e^t, exp(t)
- Trigonométricas: sin(t), cos(t), tan(t)
- Logarítmicas: log(t)
- Raíces: sqrt(t)
- Constantes: pi, e

6.4 Presentación de Resultados

Cuando se resuelve una ecuación, el panel de resultados muestra:

1. Ecuación normalizada La ecuación en forma estándar con las condiciones iniciales:

$a_2y''(t) + a_1y'(t) + a_0y(t) = f(t)$

$y(0) = c_1$

$y'(0) = c_2$

2. Transformada de Laplace La ecuación transformada en el dominio de s:

$a_2(s^2Y(s) - sy(0) - y'(0)) + a_1(sY(s) - y(0)) + a_0Y(s) = F(s)$

3. Solución en el dominio de Laplace El resultado algebraico Y(s):

Y(s) = [expresión algebraica en términos de s]

4. Solución final La función y(t) en el dominio del tiempo:

y(t) = [expresión en términos de t]

5. Gráfica de la solución Visualización de y(t) en el intervalo [0, 10]:

- Eje X: tiempo (t)
- Eje Y: valor de la función y(t)
- Cuadrícula para mejor legibilidad

6.5 Persistencia de Datos

El sistema utiliza localStorage para mantener:

- Última ecuación ingresada
- Últimas condiciones iniciales
- Preferencia de tema (claro/oscuro)
- Estado del tutorial (visto/no visto)

Esto permite que el usuario pueda:

- Cerrar y reabrir el navegador sin perder datos
- Modificar ecuaciones previas fácilmente
- Mantener su preferencia de tema

6.6 Modo Oscuro

El usuario puede alternar entre tema claro y oscuro:

- Toggle en la esquina superior derecha
- Transiciones suaves entre temas
- Preservación de preferencia en localStorage
- Paleta de colores institucionales UTEZ

6.7 Manejo de Errores

El sistema valida y reporta errores de manera amigable:

Errores de sintaxis:

"No pude interpretar la expresión: [expresión]"

Ecuaciones no lineales:

"Solo se admiten ecuaciones diferenciales lineales de primer o segundo orden"

Coeficientes variables:

"Por ahora solo se admiten coeficientes constantes en y", y' y y"

Condiciones iniciales faltantes:

"Debes capturar la condición inicial y(0)"

7. EJEMPLOS RESUELTOS

A continuación, se presentan los 4 casos de prueba requeridos con sus soluciones completas.

7.1 Caso 1: y' + 2y = 0, y(0) = 1

Tipo: Ecuación diferencial homogénea de primer orden

Entrada:

- Ecuación: y' + 2y = 0
- Condición inicial: y(0) = 1

Proceso de solución:

1. **Aplicar transformada de Laplace:**

L{y'} + 2L{y} = 0

sY(s) - y(0) + 2Y(s) = 0

sY(s) - 1 + 2Y(s) = 0

2. **Despejar Y(s):**

Y(s)(s + 2) = 1

Y(s) = 1/(s + 2)

3. **Aplicar transformada inversa:**

y(t) = e^(-2t)

Salida del programa:

- Ecuación normalizada
- Condición inicial
- Transformada
- Y(s)
- Solución final
- Gráfica

Interpretación: La solución muestra un decaimiento exponencial desde y(0) = 1, tendiendo a cero conforme t aumenta. Este comportamiento es típico de sistemas de primer orden estables.

7.2 Caso 2: y'' + 3y' + 2y = 0, y(0) = 0, y'(0) = 1

Tipo: Ecuación diferencial homogénea de segundo orden

Entrada:

- Ecuación: y'' + 3y' + 2y = 0
- Condiciones iniciales: y(0) = 0, y'(0) = 1

Proceso de solución:

1. **Aplicar transformada de Laplace:**

L{y''} + 3L{y'} + 2L{y} = 0

$s^2Y(s) - sy(0) - y'(0) + 3(sY(s) - y(0)) + 2Y(s) = 0$

$s^2Y(s) - 1 + 3sY(s) + 2Y(s) = 0$

2. **Despejar Y(s):**

$Y(s)(s^2 + 3s + 2) = 1$

$Y(s) = 1/(s^2 + 3s + 2) = 1/((s+1)(s+2))$

3. **Descomposición en fracciones parciales:**

Y(s) = 1/(s+1) - 1/(s+2)

4. **Aplicar transformada inversa:**

y(t) = e^(-t) - e^(-2t)

Interpretación: La solución es una combinación de dos exponenciales decrecientes. La gráfica muestra que y(t) crece inicialmente desde 0 (debido a y'(0) = 1), alcanza un máximo, y luego decrece hacia cero.

7.3 Caso 3: y' - y = e^t, y(0) = 1

Tipo: Ecuación diferencial no homogénea de primer orden

Entrada:

- Ecuación: $y' - y = e^t$
- Condición inicial: $y(0) = 1$

Proceso de solución:

1. **Aplicar transformada de Laplace:**

$L\{y'\} - L\{y\} = L\{e^t\}$

$sY(s) - y(0) - Y(s) = 1/(s-1)$

$sY(s) - 1 - Y(s) = 1/(s-1)$

2. **Despejar Y(s):**

$Y(s)(s - 1) = 1 + 1/(s-1)$

$Y(s) = 1/(s-1) + 1/(s-1)^2$

3. **Aplicar transformada inversa:**

$y(t) = e^t + te^t = e^t(1 + t)$

Interpretación: La presencia del término forzante e^t produce una solución que crece exponencialmente, pero además incluye un factor lineal en t. Esto genera un crecimiento más rápido que el exponencial simple.

7.4 Caso 4: $y'' + y = \sin(t)$, $y(0) = 0$, $y'(0) = 0$

Tipo: Ecuación diferencial no homogénea de segundo orden

Entrada:

- Ecuación: $y'' + y = \sin(t)$
- Condiciones iniciales: $y(0) = 0$, $y'(0) = 0$

Proceso de solución:

1. **Aplicar transformada de Laplace:**

$L\{y''\} + L\{y\} = L\{\sin(t)\}$

$s^2Y(s) - sy(0) - y'(0) + Y(s) = 1/(s^2 + 1)$

$s^2Y(s) + Y(s) = 1/(s^2 + 1)$

2. **Despejar Y(s):**

$Y(s)(s^2 + 1) = 1/(s^2 + 1)$

$Y(s) = 1/(s^2 + 1)^2$

3. **Aplicar transformada inversa:**

$y(t) = (\sin(t) - t\cdot\cos(t))/2$

Interpretación: Esta ecuación representa un oscilador armónico forzado en resonancia. La solución muestra oscilaciones con amplitud creciente linealmente con el tiempo, característica del fenómeno de resonancia.

8. REFLEXIÓN FINAL

8.1 ¿Qué dificultades se encontraron al implementar matemáticas en código?

Durante el desarrollo del proyecto, se identificaron las siguientes dificultades principales:

1. Parsing de expresiones matemáticas

- **Desafío:** Interpretar correctamente las ecuaciones ingresadas por el usuario en formato de texto libre
- **Solución:** Utilización de sympy.parsing.sympy_parser con transformaciones personalizadas
- **Aprendizaje:** La importancia de normalizar la entrada del usuario (espacios, caracteres especiales, diferentes notaciones)

2. Manejo de símbolos y sustituciones

- **Desafío:** Distinguir entre y, y', y'' como símbolos temporales durante el parsing
- **Solución:** Implementación de reemplazos estratégicos (Y0, Y1, Y2) antes del parsing, y posterior sustitución con funciones de SymPy
- **Aprendizaje:** La necesidad de mantener un flujo claro de transformación de símbolos

3. Transformada inversa de Laplace

- **Desafío:** SymPy no siempre puede calcular la transformada inversa directamente
- **Solución:** Aplicación de sp.apart() para descomponer en fracciones parciales antes de la inversión
- **Aprendizaje:** Las limitaciones de las bibliotecas simbólicas y la importancia de pre-procesar expresiones

4. Visualización numérica de soluciones simbólicas

- **Desafío:** Evaluar soluciones simbólicas que pueden contener números complejos o ser inestables numéricamente

- **Solución:** Manejo cuidadoso de sp.lambdify() con conversión a numpy y filtrado de partes imaginarias
- **Aprendizaje:** La diferencia entre cálculo simbólico y numérico, y cuándo aplicar cada uno

5. Validación de ecuaciones lineales

- **Desafío:** Garantizar que la ecuación sea lineal con coeficientes constantes
- **Solución:** Uso de sp.Poly para verificar la estructura polinomial y análisis de monomios
- **Aprendizaje:** Cómo estructurar validaciones matemáticas rigurosas en código

8.2 ¿Cómo se mejoraría el programa?

Mejoras a corto plazo:

1. **Ampliar tipos de ecuaciones soportadas**
    - Ecuaciones con coeficientes variables (no constantes)
    - Sistemas de ecuaciones diferenciales
    - Ecuaciones de orden superior (3º, 4º orden)
2. **Mejoras en la interfaz**
    - Editor de ecuaciones visual con botones para símbolos matemáticos
    - Vista paso a paso del proceso de solución
    - Historial de ecuaciones resueltas
    - Exportación de resultados a PDF
3. **Validación mejorada**
    - Sugerencias automáticas al detectar errores de sintaxis
    - Verificación de solución mediante sustitución en la ecuación original
    - Validación de estabilidad de la solución
4. **Visualización avanzada**

- Gráficas interactivas con zoom y pan
- Campo vectorial de la ecuación diferencial
- Animación de la evolución temporal
- Comparación visual con otros métodos de solución

Mejoras a largo plazo:

1. **Métodos adicionales de solución**
    - Métodos numéricos (Euler, Runge-Kutta)
    - Comparación entre método simbólico y numérico
    - Series de potencias
    - Transformada de Fourier
2. **Herramientas educativas**
    - Explicación paso a paso automática
    - Ejercicios generados aleatoriamente
    - Sistema de pistas y ayudas
    - Evaluación automática de respuestas
3. **Optimización del código**
    - Caché de resultados para ecuaciones repetidas
    - Procesamiento asíncrono para ecuaciones complejas
    - Mejora de rendimiento en cálculos numéricos
4. **Accesibilidad**
    - Soporte para lectores de pantalla
    - Atajos de teclado
    - Modo de alto contraste
    - Internacionalización (i18n)

9. CONCLUSIONES

9.1 Logros del Proyecto

El proyecto ha alcanzado exitosamente todos los objetivos propuestos:

- **Implementación funcional**: Se desarrolló una aplicación web completa capaz de resolver ecuaciones diferenciales de primer y segundo orden mediante transformada de Laplace
- **Interfaz intuitiva**: Se creó una interfaz web moderna y amigable que facilita la interacción del usuario con conceptos matemáticos complejos
- **Robustez**: El sistema maneja correctamente errores de entrada, proporciona mensajes claros y valida las ecuaciones antes de procesarlas
- **Visualización**: Se integró la generación automática de gráficas y el renderizado de ecuaciones en formato LaTeX
- **Casos de prueba**: Los 4 casos mínimos requeridos se resuelven correctamente, demostrando la validez del método implementado

9.2 Competencias Adquiridas

A través del desarrollo de este proyecto, se consolidaron las siguientes competencias:

Técnicas:

- Programación en Python con enfoque modular
- Desarrollo web con Flask
- Manipulación simbólica con SymPy
- Visualización de datos con Matplotlib
- Diseño de interfaces con HTML/CSS/JavaScript

Matemáticas:

- Comprensión profunda de la transformada de Laplace
- Análisis de ecuaciones diferenciales ordinarias
- Validación matemática de soluciones

- Interpretación de resultados simbólicos y numéricos

Ingeniería de software:

- Diseño de arquitectura MVC
- Manejo de errores y validación de entrada
- Documentación técnica
- Testing de funcionalidades

9.3 Aplicaciones Prácticas

Este tipo de herramienta tiene aplicaciones en diversos campos:

- **Educación**: Como herramienta didáctica para estudiantes de ingeniería y ciencias
- **Ingeniería**: Análisis de circuitos eléctricos, sistemas mecánicos, control automático
- **Física**: Modelado de sistemas dinámicos, oscilaciones, ondas
- **Investigación**: Prototipado rápido de soluciones a problemas de ecuaciones diferenciales

9.4 Trabajo Futuro

El proyecto establece una base sólida para futuras extensiones. Las líneas de trabajo futuro incluyen:

1. Ampliación del rango de ecuaciones soportadas
2. Integración de métodos numéricos complementarios
3. Desarrollo de un módulo educativo con explicaciones paso a paso
4. Implementación de un sistema de ejercicios con evaluación automática
5. Optimización para dispositivos móviles
6. Creación de una API REST para integración con otras aplicaciones

CRÉDITOS

Equipo de desarrollo:

- Raúl Corcino León
- Andrés Hernández Popoca
- Bryan Venegas Sánchez
- Alejandro Castelar Hernández
- Oscar Rosales García

Institución: Universidad Tecnológica Emiliano Zapata del Estado de Morelos (UTEZ) Academia de Ciencias

Fecha de desarrollo: 2026